

TP3-1

December 8, 2025

1 TP 3 - Grupo 7

André Filipe Dourado Pinheiro - A108473

Tiago Silva Costa - A108657

1.1 Problema 1 - Algoritmo estendido de Euclides

O algoritmo estendido de Euclides (EXA) aceita dois inteiros constantes $a, b > 0$ e devolve inteiros r, s, t tais que $a * s + b * t = r$ e $r = \gcd(a, b)$. Para além das variáveis r, s, t o código requer 3 variáveis adicionais r', s', t' que representam os valores de r, s, t no “próximo estado”.

```
INPUT  a, b
assume  a > 0 and b > 0
r, r', s, s', t, t' = a, b, 1, 0, 0, 1
while r' != 0
    q = r div r'
    r, r', s, s', t, t' = r', r - q * r', s', s - q * s', t', t - q * t'
OUTPUT r, s, t
```

1. Construa um SFOTS usando BitVector's de tamanho $n = 16$ bits que descreva o comportamento deste programa. Considere estado de erro quando $r = 0$ ou alguma das variáveis atinge o “overflow”.
2. Usando a metodologia das “Constraint Horn Clauses”(chc's) verifique se é possível determinar um invariante que garanta que nunca se atinge um estado de erro.
3. Verifique, usando a metodologia dos invariantes e interpolantes, se o modelo atinge um estado de erro. Para o cálculo do interpolante usar a metodologia das “Constraint Horn Clauses”(chc's).

1.2 Modelação

Com o objetivo de resolver o problema proposto, optou-se pela utilização do módulo `z3`. Além disso, iremos utilizar o módulo `itertools`, que será relevante para o *model-checking*.

```
[1]: import z3
from z3 import *
import itertools
```

1.3 Implementação

1.3.1 Construção do SFOTS

Vamos construir o SFOTS usando uma *BitVector's* de tamanho $n = 16$ que descreva o comportamento do programa. Para isso, vamos recordar a definição de um SFOTS, que é dado pelo seguinte tuplo:

$$\Sigma \equiv \langle \mathcal{T}, X, I, T, P \rangle$$

Onde se verifica o seguinte, para representar o sistema em específico: 1. $\mathcal{T}$ representa um SMT apropriada, que pertence à FOL, que vamos representar no nosso Solver; 2. X é o conjunto das variáveis base do problema; 3. $next$ é um operador que gera “clones” das variáveis em X ; 4. I é um predicado unário que determina quais os estados iniciais; 5. T é um predicado binário que determina as transições entre dois estados; 6. E é um predicado unário que determina os estados de erro.

Nesse sentido, o modelo terá 6 variáveis do tipo BitVector, r, r', s, s', t, t' . Para além destes, também terá uma variável inteira pc que representa o estado de execução. Definiu-se um inteiro para cada um dos estados, nomeadamente: - 1 para Estado central (loop). - 0 para Estado Final (fora do loop).

As cópias destas variáveis serão dadas pelo operador $next$, cuja notação se pode expandir para incluir qualquer predicado P que tenha X como o conjunto das variáveis livres. Assim, $next(X) \equiv X'$ e $next(P) \equiv P' \equiv P\{X / next(X)\}$. Esta notação segue a convenção das aulas teóricas, o que lhe permite ficar “livre de variáveis”.

Considerando o estado inicial, vamos otimizar a execução do autómato definindo $b < a$. Isto diminui o número de operações totais necessárias, no pior caso. O estado inicial será então definido pelo predicado seguinte:

$$I \equiv (r \geq r') \wedge (r = a) \wedge (r' = b) \wedge (s = 1) \wedge (s' = 0) \wedge (t = 0) \wedge (t' = 1) \wedge (pc = 1)$$

O predicado de transição terá a seguinte forma.

$$T \equiv \begin{aligned} & (pc = 1 \wedge r' \neq 0 \wedge pc_{-} = 1 \wedge r_{-} = r' \wedge r'_{-} = r \pmod{2} \wedge s_{-} = s' \wedge s'_{-} = s - q \times s' \wedge t_{-} = t' \wedge t'_{-} = t - q \times \\ & \quad \vee \\ & (pc = 1 \wedge r' = 0 \wedge pc_{-} = 0 \wedge r_{-} = r \wedge r'_{-} = r \wedge s_{-} = s \wedge s'_{-} = s' \wedge t_{-} = t \wedge t'_{-} = t') \end{aligned}$$

Como condição de erro, considera-se o caso em que $r = 0$ e também quando ocorre *overflow*, ou seja, tem-se

$$E \equiv (r = 0) \vee (\text{overflow})$$

[2]: N = 16

Para gerar os estados, a função `genState(vars_names, prefix, i)` recebe as variáveis do problema e, para cada uma, cria-se um *Bitvector*.

```
[3]: def genState(vars_names, prefix, i):  
    state = {}  
    for v in vars_names:  
        state[v] = BitVec(f"{prefix}_{v}_{i}", N)  
    return state
```

As variáveis do problema são r, s, t, r', s', t' e pc , todos *BitVectors*.

```
[4]: var_names = ['r', 's', 't', 'rp', 'sp', 'tp', 'pc']
```

A função `init1(state)` devolve o predicado do estado inicial.

```
[5]: def init1(state):  
    r, rp = state['r'], state['rp']  
    s, sp = state['s'], state['sp']  
    t, tp = state['t'], state['tp']  
    pc = state['pc']  
  
    return And(  
        r > 0, rp > 0,  
        r >= rp,  
        s == 1, sp == 0,  
        t == 0, tp == 1,  
        pc==1,  
    )
```

Definimos também `trans1`, que retorna o predicado final das transições.

```
[6]: def trans1(curr, nxt):  
    # Estado Atual  
    r, rp = curr['r'], curr['rp']  
    s, sp = curr['s'], curr['sp']  
    t, tp = curr['t'], curr['tp']  
    pc = curr['pc']  
  
    # Próximo Estado  
    r_, rp_ = nxt['r'], nxt['rp']  
    s_, sp_ = nxt['s'], nxt['sp']  
    t_, tp_ = nxt['t'], nxt['tp']  
    pc_ = nxt['pc']  
  
    # Lógica auxiliar: Divisão inteira (unsigned para r e rp, pois são  
    ↪ positivos)  
    q = r/rp
```

```

# Transição 1: Passo do Algoritmo (Loop)
t_step = And(
    pc == 1,
    rp != 0,

    pc_ == 1,
    r_ == rp,
    rp_ == r%rp,
    s_ == sp,
    sp_ == s - q*sp,
    t_ == tp,
    tp_ == t - q*tp,
    pc_ == pc
)

# Transição 2: Parada (Stop)
t_stop = And(
    pc == 1,
    rp == 0,
    r_ == r, rp_ == rp,
    s_ == s, sp_ == sp,
    t_ == t, tp_ == tp,
    pc_ == 0
)

return Or(t_step, t_stop)

```

Para definirmos o estado de erro, vamos criar a seguinte função auxiliar `coverflow(val, val_prime, q)` que será responsável por verificar se ocorre *overflow* na realização do cálculo $val - (q * val_prime)$. Isto é feito através da extensão do sinal e verificando se o cálculo final encontra-se *bounded*.

```

[7]: def coverflow(val, val_prime, q):
    val_ext = SignExt(32-N, val)
    valp_ext = SignExt(32-N, val_prime)
    q_ext = SignExt(32-N, q)

    res = val_ext - valp_ext * q_ext

    return Or(res > 2**(N-1)-1, res < -2**(N-1))

```

Por fim, definimos a função `error1(state)` que devolve o predicado do estado de erro.

```

[8]: def error1(state):
    r, rp = state['r'], state['rp']
    s, sp = state['s'], state['sp']
    t, tp = state['t'], state['tp']
    pc = state['pc']

```

```

q = r/rp

return And(pc==1, rp > r, Or(r == 0,coverflow(s,sp,q), coverflow(t,tp,q)))

```

A função `genTrace` recebe todas estas funções, exceto a do predicado do erro, mais o número `n`, tamanho máximo do traço, e devolve um traço execução sem qualquer consideração pelo estado de erro.

```

[9]: def genTrace(vars, init, trans, n):
    with Solver() as s:
        # Cria n+1 estados
        X = [genState(vars, 'X', i) for i in range(n + 1)]

        # Aplica Init no estado 0.
        I = init(X[0])

        # Cria lista de restrições de transição
        Tks = [trans(X[i], X[i+1]) for i in range(n)]

        # Resolve I /\ T~n
        if s.check(I, And(Tks)) == sat:
            m = s.model()
            for i in range(n + 1):
                print(f"Estado: {i}")
                for v in vars:
                    # Pega o valor do modelo
                    val_z3 = m[X[i][v]]
                    # Converte para int (as_signed_long para ver negativos
                    ↪corretamente)
                    val_int = val_z3.as_signed_long()
                    print(f"        {v} = {val_int}")
                print("-" * 20)
            else:
                print("UNSAT: Traço não encontrado.")

```

Por fim, podemos gerar um traço qualquer para verificar se o SFOTS está implementado corretamente.

```

[10]: genTrace(var_names, init1, trans1, 15)

```

```

Estado: 0
r = 30437
s = 1
t = 0
rp = 19494
sp = 0
tp = 1
pc = 1

```

Estado: 1
 r = 19494
 s = 0
 t = 1
 rp = 10943
 sp = 1
 tp = -1
 pc = 1

Estado: 2
 r = 10943
 s = 1
 t = -1
 rp = 8551
 sp = -1
 tp = 2
 pc = 1

Estado: 3
 r = 8551
 s = -1
 t = 2
 rp = 2392
 sp = 2
 tp = -3
 pc = 1

Estado: 4
 r = 2392
 s = 2
 t = -3
 rp = 1375
 sp = -7
 tp = 11
 pc = 1

Estado: 5
 r = 1375
 s = -7
 t = 11
 rp = 1017
 sp = 9
 tp = -14
 pc = 1

Estado: 6
 r = 1017

```
s = 9
t = -14
rp = 358
sp = -16
tp = 25
pc = 1
```

Estado: 7

```
r = 358
s = -16
t = 25
rp = 301
sp = 41
tp = -64
pc = 1
```

Estado: 8

```
r = 301
s = 41
t = -64
rp = 57
sp = -57
tp = 89
pc = 1
```

Estado: 9

```
r = 57
s = -57
t = 89
rp = 16
sp = 326
tp = -509
pc = 1
```

Estado: 10

```
r = 16
s = 326
t = -509
rp = 9
sp = -1035
tp = 1616
pc = 1
```

Estado: 11

```
r = 9
s = -1035
t = 1616
rp = 7
```

```

        sp = 1361
        tp = -2125
        pc = 1
-----
Estado: 12
        r = 7
        s = 1361
        t = -2125
        rp = 2
        sp = -2396
        tp = 3741
        pc = 1
-----
Estado: 13
        r = 2
        s = -2396
        t = 3741
        rp = 1
        sp = 8549
        tp = -13348
        pc = 1
-----
Estado: 14
        r = 1
        s = 8549
        t = -13348
        rp = 0
        sp = -19494
        tp = 30437
        pc = 1
-----
Estado: 15
        r = 1
        s = 8549
        t = -13348
        rp = 0
        sp = -19494
        tp = 30437
        pc = 0
-----

```

Verificando os valores, podemos ver que SFOTS funciona corretamente.

1.4 Metodologia das “Constraint Horn Clauses” (chc’s)

Queremos agora utilizar a metodologia das “Constraint Horn Clauses”(chc’s) para verificar se é possível determinar um invariante que garanta que nunca se atinge um estado de erro.

Para isso, vamos utilizar as seguintes funções auxiliares que foram implementadas nas aulas práticas.


```

[11]: class HtmlStr(object):
    def __init__(self, s):
        self._s = str(s)
        self._s = self._s.replace('\n', '<br/>')

    def _repr_html_(self):
        return self._s

    def __repr__(self):
        return repr(self._s)
    def __str__(self):
        return str(self._s)

def chc_to_str(chc):
    if z3.in_html_mode():
        return chc_to_html(chc)
    else:
        return chc_to_txt(chc)

def chc_to_html(chc):
    import io
    out = io.StringIO()

    for cls in chc:
        print(cls, '<br/>', file=out)

    return HtmlStr(out.getvalue())

def chc_to_txt(chc):
    import io
    out = io.StringIO()
    for cls in chc:
        print(cls, file=out)
    return out.getvalue()

class SpacerProof(object):
    def __init__(self, pf_ast):
        # strip off last mp to false
        self._ast = pf_ast.children()[0]

    def _get_fact(self, ast):
        return ast.children()[-1]

    def _to_dot_rec(self, ast, graph, visited):
        if ast.get_id() in visited:
            return

```

```

        visited.add(ast.get_id())

    dst = str(self._get_fact(ast).get_id())
    kids = ast.children()
    for k in kids[1:-1]:
        k_fact = self._get_fact(k)
        if k_fact.get_id() not in visited:
            graph.node(str(k_fact.get_id()), str(k_fact))
            visited.add(k_fact.get_id())
            self._to_dot_rec(k, graph, visited)
            graph.edge(str(k_fact.get_id()), dst)

    def to_dot(self):
        import graphviz #Instalar no pack manager
        g = graphviz.Digraph()

        visited = set()

        fact = self._get_fact(self._ast)
        id = fact.get_id()
        g.node(str(id), str(fact))
        visited.add(id)
        self._to_dot_rec(self._ast, g, visited)
        return g

    def _repr_mimebundle_(self, include, exclude, **kwargs):
        return self.to_dot()._repr_mimebundle_(include, exclude, **kwargs)

    def __str__(self):
        return str(self.to_dot())
    def raw(self):
        return self._ast

# proof mode must be enabled before any expressions are created
z3.set_param(proof=True)
z3.set_param(model=True)
# print expressions with HTML
z3.set_html_mode(True)

# wrapper to solve CHC constraints and extract result
def solve_horn(chc, pp=False, q3=False, max_unfold=10, verbosity=0):
    z3.set_param(verbose=verbosity)

    s = z3.SolverFor('HORN')
    s.set('engine', 'spacer')
    s.set('spacer.order_children', 2)

```

```

if not pp:
    s.set('xform.inline_eager', False)
    s.set('xform.inline_linear', False)
    s.set('xform.slice', False)

if max_unfold > 0:
    s.set('spacer.max_level', max_unfold)

if q3:
    # allow quantified variables in pobs
    s.set('spacer.ground_pobs', False)
    # enable quantified generalization
    s.set('spacer.q3.use_qgen', True)

# add constraints to solver
s.add(chc)
if verbosity > 0:
    print(s.sexpr())
# run solver
res = s.check()
# extract model or proof
answer = None
if res == z3.sat:
    answer = s.model()
elif res == z3.unsat:
    answer = s.proof()
return res, answer

class Ts(object):
    """A transition system

    Example usage
    >>> T = Ts('Ts0')
    >>> x, x_out = T.add_var(z3.IntSort(), name='x')
    >>> T.Init = x <= 0
    >>> T.Tr = z3.And(x < 5, x_out == x + 1)
    >>> T.Bad = x >= 10
    >>> T
    Transition System: Ts0
        Init: v_0 <= 0
        Bad: v_0 >= 10
        Tr: And(v_0 < 5, v_out_0 == v_0 + 1)
    """
    def __init__(self, name='Ts'):
        # string name
        self.name = name
        # state variables (pair of input and output)

```

```

self._vars = []
# inputs
self._inputs = []
# a map from optional names to state variables
self._named_vars = dict()

# maps state variable index to optional name
self._var_names = list()

# Transition relation
self.Tr = z3.BoolVal(True)
# Initial condition
self.Init = z3.BoolVal(True)
# Bad states
self.Bad = z3.BoolVal(False)

def add_var(self, sort, name=None):
    '''Add a state variable of a given sort. Returns a pair (pre, post)
    of a pre- and post- state versions of the variable
    '''
    pre, post = self._new_var_name(name=name)
    v_in = z3.Const(pre, sort)
    v_out = z3.Const(post, sort)
    self._vars.append((v_in, v_out))
    self._var_names.append(name)
    if name is not None:
        self._named_vars[name] = (v_in, v_out)

    return (v_in, v_out)

def get_var(self, idx):
    """Returns a pair of pre- and post-variables with a given index or name

    If idx is not an int it is interpreted as a name.
    Otherwise, it is interpreted as a variable index.

    >>> T = Ts('Ts0')
    >>> x, x_out = T.add_var(z3.IntSort(), name='x')
    >>> y, y_out = T.add_var(z3.IntSort(), name='y')
    >>> x
    v_0

    >>> T.get_var(1)
    (v_1, v_out_1)

    >>> T.get_var('x')

```

```

        (v_0, v_out_0)

    """
    if isinstance(idx, int):
        return self._vars[idx]
    elif idx in self._named_vars:
        return self._named_vars[idx]
    return None

def get_var_name(self, idx):
    if idx < len(self._var_names):
        return self._var_names[idx]
    return None

def get_pre_var(self, idx):
    """Returns a pre-state variable with a given name/index"""
    res = self.get_var(idx)
    if res is not None:
        return res[0]
    return None

def get_pre_vars(self, vars):
    """Returns a tuple of pre-state variables with given names"""
    return (self.get_pre_var(v) for v in vars.split())

def get_post_var(self, idx):
    """Returns a post-state variable with a given name"""
    res = self.get_var(idx)
    if res is not None:
        return res[1]
    return None

def add_input(self, sort, name=None):
    '''Add an input of a given sort'''
    v = z3.Const(self._new_input_name(name=name), sort)
    self._inputs.append(v)
    return v

def inputs(self):
    return self._inputs
def pre_vars(self):
    return [u for (u, v) in self._vars]
def post_vars(self):
    return [v for (u, v) in self._vars]
def vars(self):
    return self.pre_vars() + self.post_vars()
def pre_post_vars(self):

```

```

        return self._vars
def all(self):
    return self.vars() + self.inputs()
def sig(self):
    return [v.sort() for (u, v) in self._vars]
def to_post(self, e):
    '''Rename expression over pre-state variables to post-state variables

    >>> T = Ts('Ts0')
    >>> x, x_out = T.add_var(z3.IntSort(), 'x')
    >>> y, y_out = T.add_var(z3.IntSort(), 'y')
    >>> e = x > y
    >>> T.to_post(x > y)
    v_out_0 > v_out_1
    '''

    return z3.substitute(e, *self._vars)

def _new_input_name(self, name=None):
    if name is not None:
        return str(name)
    else:
        return self._mk_input_name(len(self._inputs))

def _mk_input_name(self, idx):
    return 'i_' + str(idx)

def _new_var_name(self, name=None):
    if name is not None:
        assert name not in self._named_vars
        assert str(name) not in self._named_vars
        return str(name), str(name) + ""
    else:
        idx = len(self._vars)
        return self._mk_var_name(idx), self._mk_post_var_name(idx)

def _mk_var_name(self, idx):
    return 'v_' + str(idx)
def _mk_post_var_name(self, idx):
    return 'v_out_' + str(idx)

def __repr__(self):
    return 'Transition System: ' + self.name + '\n' + \
        '\tInit: ' + str(self.Init) + '\n' + \
        '\tBad: ' + str(self.Bad) + '\n' + \
        '\tTr: ' + str(self.Tr)

def __str__(self):

```

```

        return repr(self)

def vc_gen(T):
    Inv = z3.Function('Inv', *(T.sig() + [z3.BoolSort()])))

    InvPre = Inv(*T.pre_vars())
    InvPost = Inv(*T.post_vars())

    all_vars = T.all()
    vc_init = z3.ForAll(all_vars, z3.Implies(T.Init, InvPre))
    vc_ind = z3.ForAll(all_vars, z3.Implies(z3.And(InvPre, T.Tr), InvPost))
    vc_bad = z3.ForAll(all_vars, z3.Implies(z3.And(InvPre, T.Bad), z3.
↪ BoolVal(False)))

    return [vc_init, vc_ind, vc_bad], InvPre

```

Para aplicar esta metodologia, comecemos por implementar um *Transition System* com os mesmos predicados definidos anteriormente, isto é, criamos um objeto do tipo Ts e adicionamos as variáveis necessárias, estado inicial, transições e estado de erro.

```

[12]: T = Ts('EXA')
r, r_ = T.add_var(BitVecSort(N), name='r')
s, s_ = T.add_var(BitVecSort(N), name='s')
t, t_ = T.add_var(BitVecSort(N), name='t')
pc, pc_ = T.add_var(BoolSort(), name='pc')

rp, rp_ = T.add_var(BitVecSort(N), name='rp')
sp, sp_ = T.add_var(BitVecSort(N), name='sp')
tp, tp_ = T.add_var(BitVecSort(N), name='tp')

T.Init = And(
    r > 0, rp > 0,
    r>rp,
    s == 1, sp == 0,
    t == 0, tp == 1,
    pc
)

q = r / rp

t_step = And(
    pc,
    rp > 0,
    r>rp,

    r_ == rp,
    rp_ == r%rp,

```

```

    s_ == sp,
    sp_ == s - q*sp,
    t_ == tp,
    tp_ == t - q*tp,
    pc_ == pc
)

t_stop = And(
    pc,
    rp == 0,
    r_ == r,
    rp_ == rp,
    s_ == s,
    sp_ == sp,
    t_ == t,
    tp_ == tp,
    pc_ == Not(pc)
)

T.Tr = Or(t_step, t_stop)
T.Bad = And(pc, r == 0, rp>=r, coverflow(s,sp,q), coverflow(t,tp,q))

```

Por fim, realizamos uma chamada à função `vc_gen()`, que retornará uma lista com os predicados necessários relacionados as CHCs e também a função do invariante.

Após isso, realizamos uma chamada à função `solve_horn()`, que será responsável por utilizar um solver com o algoritmo Spacer para encontrar o invariante.

Em seguida, caso o invariante seja encontrado, fazemos *print* do mesmo.

```

[13]: vc, inv = vc_gen(T)
      res, ans = solve_horn(vc)
      ans.eval(inv)

```

```

[13]: &not;(r &le; rp)

```

Ora, do invariante que foi retornado, temos

$$r' < r$$

que de facto corresponde ao que era desejado.

1.5 Metodologia dos invariantes e interpolantes

Queremos agora, usando a metodologia dos invariantes e interpolantes, verificar se o modelo atinge um estado de erro. Para isso, o cálculo do interpolante vai utilizar a metodologia das *Constraint Horn Clauses* (chc's).

Começemos por definir algumas funções auxiliares que foram implementadas durante as aulas práticas.


```

[14]: def free_arith_vars(fml):
    seen = set([])
    vars = set([])

    bitvec_sort = z3.BitVecSort(N)

    def fv(seen, vars, f):
        if f in seen:
            return
        seen |= { f }
        if f.sort().eq(bitvec_sort) and f.decl().kind() == z3.
↳ Z3_OP_UNINTERPRETED:
            vars |= { f }
        for ch in f.children():
            fv(seen, vars, ch)

    fv(seen, vars, fml)

    return vars

def interpolate(A, B):

    As = free_arith_vars(A)
    Bs = free_arith_vars(B)

    shared = [s for s in As & Bs]

    print(As, Bs)

    Itp = z3.Function('Itp', [s.sort() for s in shared] + [z3.BoolSort()])
    left = z3.ForAll([a for a in As], z3.Implies(A, Itp(shared)))
    right = z3.ForAll([b for b in Bs], z3.Implies(Itp(shared), z3.Not(B)))

    res, ans = solve_horn([left, right])

    if res == z3.sat:
        return ans.eval(Itp(shared))

    return None

```

Além disso, vamos criar a função `init_ab(state, a, b)`, que permitirá criar um estado inicial com valores customizados de *a* e *b*.

```

[15]: def init_ab(state, a, b):
    r, rp = state['r'], state['rp']
    s, sp = state['s'], state['sp']
    t, tp = state['t'], state['tp']

```

```

pc = state['pc']

return And(
    r == a, rp == b,
    r >= rp,
    s == 1, sp == 0,
    t == 0, tp == 1,
    pc==1,
)

```

Finalmente, definimos a seguinte implementação do algoritmo de model-checking semelhante ao que foi realizado nas aulas práticas, nomeadamente da resolução da *ficha9*. A diferença particular é a definida pelo enunciado, onde o incremento ao n e ao m é feito por interpolação ao utilizador.

```

[16]: def baseName(s):
    """Remove o sufixo '!' gerado pelo interpolante do Z3."""
    return ''.join(list(itertools.takewhile(lambda x: x != '!', s)))

def rename(form, state):
    """Substitui variáveis pelo estado correspondente."""
    vs = list(form.children()) if not is_const(form) else [form]
    mapping = {}
    for v in vs:
        if is_const(v) and v.decl().kind() == Z3_OP_UNINTERPRETED:
            b = baseName(v.decl().name())
            if b in state:
                mapping[v] = state[b]
    if mapping:
        return substitute(form, *[(k, v) for k, v in mapping.items()])
    return form

def same(state1, state2):
    """Igualdade ponto a ponto entre estados."""
    return And(*[state1[x] == state2[x] for x in state1])

def model_checking(vars, init, trans, error, N, M, a, b):

    s = Solver()

    # Estados forward X e backward Y
    X = [genState(vars, "X", i) for i in range(N+1)]
    Y = [genState(vars, "Y", i) for i in range(M+1)]

    # Passo n=0, m=0
    I = init_ab(X[0], a, b)

```

```

E = error(Y[0])

if s.check(I, E, same(X[0], Y[0])) == sat:
    print("Unsafe!")
    return

n = 1
m = 1

# Loop principal
while n <= N and m <= M:
    print("n =", n, "m =", m)

    Tn = And(*[trans(X[i], X[i+1]) for i in range(n)])
    Bm = And(*[trans(Y[j+1], Y[j]) for j in range(m)]) # inverso

    Rn = And(I, Tn)
    Um = And(E, Bm)

    Vnm = And(Rn, Um, same(X[n], Y[m]))

    # 1) Checar contra-exemplo
    if s.check(Vnm) == sat:
        print("Unsafe!")
        return

    # 2) Interpolante binário
    C = interpolate(And(Rn, same(X[n], Y[m])), Um)
    if C is None:
        print("Interpolante None!")
        continue

    # 3) Testar se é invariante
    C0 = rename(C, X[0])
    C1 = rename(C, X[1])
    T = trans(X[0], X[1])

    if s.check(C0, T, Not(C1)) == unsat:
        print("Safe: interpolante é invariante!")
        return

    # 4) Expandir majorante S
    S = rename(C, X[n])

    while True:
        A = And(S, trans(X[n], Y[m]))

```

```

    if s.check(A, Um) == sat:
        print("Interpolante None!")
        break

    Cnew = interpolate(A, Um)
    Cn = rename(Cnew, X[n])

    # Verifica se Cn adiciona nova informação
    if s.check(Cn, Not(S)) == sat:
        S = Or(S, Cn)
    else:
        print("Safe: majorante é invariante!")
        return

    # interação com o utilizador
    nm = input("Escolha quem incrementar (n/m): ")
    inc = int(input("Incremento: "))

    if nm == "n":
        n += inc
    else:
        m += inc

print("unknown")

```

Finalmente, podemos verificar com alguns exemplos

```
[17]: model_checking(var_names, init1, trans1, error1, 50, 50,60000,2)
```

```

n = 1 m = 1
{Y_pc_1, Y_rp_1, X_tp_0, X_pc_0, X_sp_1, Y_sp_1, X_r_1, X_sp_0, X_t_0, X_rp_1,
Y_s_1, Y_r_1, X_s_1, X_pc_1, X_s_0, X_r_0, X_t_1, X_tp_1, X_rp_0, Y_t_1, Y_tp_1}
{Y_pc_1, Y_rp_1, Y_s_0, Y_s_1, Y_r_1, Y_sp_1, Y_sp_0, Y_t_0, Y_rp_0, Y_tp_1,
Y_t_1, Y_r_0, Y_tp_0, Y_pc_0}
Safe: interpolante é invariante!

```

```
[18]: model_checking(var_names, init1, trans1, error1, 10, 30, 82,100)
```

```

n = 1 m = 1
{Y_pc_1, Y_rp_1, X_tp_0, X_pc_0, X_sp_1, Y_sp_1, X_r_1, X_sp_0, X_t_0, X_rp_1,
Y_s_1, Y_r_1, X_s_1, X_pc_1, X_s_0, X_r_0, X_t_1, X_tp_1, X_rp_0, Y_t_1, Y_tp_1}
{Y_pc_1, Y_rp_1, Y_s_0, Y_s_1, Y_r_1, Y_sp_1, Y_sp_0, Y_t_0, Y_rp_0, Y_tp_1,
Y_t_1, Y_r_0, Y_tp_0, Y_pc_0}
Safe: interpolante é invariante!

```

```
[ ]: model_checking(var_names, init1, trans1, error1, 2, 2, 28657, 17711)
```

```
n = 1 m = 1
```

```

{Y_pc_1, Y_rp_1, X_tp_0, X_pc_0, X_sp_1, Y_sp_1, X_r_1, X_sp_0, X_t_0, X_rp_1,
Y_s_1, Y_r_1, X_s_1, X_pc_1, X_s_0, X_r_0, X_t_1, X_tp_1, X_rp_0, Y_t_1, Y_tp_1}
{Y_pc_1, Y_rp_1, Y_s_0, Y_s_1, Y_r_1, Y_sp_1, Y_sp_0, Y_t_0, Y_rp_0, Y_tp_1,
Y_t_1, Y_r_0, Y_tp_0, Y_pc_0}
Interpolante None!
n = 1 m = 1
{Y_pc_1, Y_rp_1, X_tp_0, X_pc_0, X_sp_1, Y_sp_1, X_r_1, X_sp_0, X_t_0, X_rp_1,
Y_s_1, Y_r_1, X_s_1, X_pc_1, X_s_0, X_r_0, X_t_1, X_tp_1, X_rp_0, Y_t_1, Y_tp_1}
{Y_pc_1, Y_rp_1, Y_s_0, Y_s_1, Y_r_1, Y_sp_1, Y_sp_0, Y_t_0, Y_rp_0, Y_tp_1,
Y_t_1, Y_r_0, Y_tp_0, Y_pc_0}

```